

System Integration & Architecture Manual

Project: GitAnalyzer Distributed Pipeline

Environment: Local Development (Spring Boot, Docker PostgreSQL, n8n Automation)

Author: Backend Systems Engineering Team

1. Architectural Blueprint: System Components

Our end-to-end architecture operates as a **Decoupled, Event-Driven REST Pipeline**. Here is the precise definition of each component in your local stack:

A. Thunder Client (The Client / Trigger)

- **Definition:** A lightweight API testing extension executed entirely within the VS Code environment.
- **Core Function:** It stands in place of a front-end user interface during the backend isolation development phase. It constructs raw network sockets, formats input into standard HTTP packets, validates JSON bodies, and fires the payload to the server gateway.

B. Spring Boot Application (The Business Logic Engine)

- **Definition:** A Java enterprise framework running on port 8081 handling web traffic routing, dependency injection, and data orchestration.
- **Core Function:** It exposes a structured endpoint via the RepositoryController. Once hit, it unmarshalls incoming JSON into strongly typed Java request objects, coordinates transaction boundaries via the service layer, and hands tasks off to downstream components.

C. PostgreSQL Container with pgvector (The Persistence Tier)

- **Definition:** An isolated relational database engine executing inside a detached Docker container on port 5432.
- **Core Function:** Managed via Hibernate's Object-Relational Mapping (ORM) framework, it acts as the project's permanent ledger, generating structured schemas, tracking system configurations, and saving relational entities.

D. n8n Webhook Node (The Workflow Automation Target)

- **Definition:** An open-source, node-based automation platform running on port 5678.
- **Core Function:** The Webhook Node acts as a passive network socket listener. It waits on an open state (/webhook-test/...) to receive external POST payloads, immediately ingesting and exposing data fields to downstream automation or AI nodes.

2. The Complete Data Lifecycle Chain

When you click "**Send**" inside your client console, the data transfers along a strict, sequential relay chain across four isolated ports:

Step 1: Client Transit

Thunder Client intercepts your workspace parameters and transmits an outbound HTTP POST packet directly to the local network bridge target:
`http://localhost:8081/api/repositories/analyze`.

Step 2: Gateway Mapping

The embedded Apache Tomcat server inside Spring Boot intercepts the packet on port 8081. It activates the **RepositoryController**, matching the incoming URI against your explicit `@PostMapping("/analyze")` annotation mapping rule.

Step 3: Service Layer Delegation

The controller routes the extracted parameters directly into **RepositoryService.java**. Here, the service acts as a manager, dividing tasks between two completely separate specialized components:

- **The Repository Layer (JpaRepository):** Receives the entity definitions, executes raw internal database drivers, communicates with PostgreSQL on port 5432, and creates the rows. It attaches a freshly generated unique repold (UUID) to the object.
- **The HTTP Client (RestTemplate):** Captures the updated object containing the newly created database tracker token and prepares a new outbound request bundle.

Step 4: Webhook Trigger

RestTemplate constructs an independent web call, maps the data back into a serialized JSON structure, and initiates an HTTP call outward to your running Docker engine on port 5678. n8n's active socket catches it, registers the incoming parameters, and updates its visual UI checkmark to **Green (200 Success)**.

3. Core Utility Definition: What is RestTemplate?

RestTemplate is a **specific, built-in library class** provided explicitly by the core Spring Framework (`org.springframework.web.client.RestTemplate`).

- **The Core Mechanism:** It is an internal HTTP web client engine. Just like your browser or Thunder Client lets a human click buttons to access a web page, RestTemplate lets a Java application programmatically talk to outer servers using raw web protocols (GET, POST, PUT, DELETE).
- **The Lifecycle Isolation:** It has zero visibility into databases, SQL tables, or data sheets. Its sole purpose is to translate internal Java structures into stringified JSON and push them across standard network web gateways.

4. Production Roadmap: Moving Beyond Testing

Right now, you are utilizing **Decoupled Development Architecture**. This structure isolates parts of the system to verify stability early. When your development cycle transitions from testing into production, the architecture scales smoothly without altering your core server configurations:

Component	Current Testing Phase	Final Production Phase
-----------	-----------------------	------------------------

Component	Current Testing Phase	Final Production Phase
Trigger Action	Thunder Client UI Input Panels	Frontend UI web components
API Target Path	http://localhost:8081/api/repositories/analyze	Same underlying endpoint, triggered via integrated frontend fetch() statements
Database Sync	Automated local updates to verify mappings	Real-time live production database instance persistence